Gantt Page 50620

Technical Document

How the BC page 'Gantt Demo DHX 2', the control add-in 'DHX Gantt Control 2', wrapper.js and the DHTMLX Gantt component work together

Object	Page 50620 'Gantt Demo DHX 2' + control add-in 'DHX Gantt Control 2'
Scope	AL page and triggers, add-in contract, wrapper.js (every function and variable), data build in AL, task panel, resource panel, menus, refresh and navigation, settings, actions
Source folder	src/dhx/ganttdemo2/
Generated	03 November 2026
Generator	docs/generate_gantt_50620_doc.py (content in docs/gantt_doc/)

Contents

1. Purpose, scope and file inventory 5

1.1 What the page does . . . 5

1.2 Source files . . . 5

2. Architecture overview 6

2.1 Three rules of this integration . . . 6

2.2 End-to-end data flow (read path) . . . 6

2.3 End-to-end data flow (write path) . . . 7

3. The control add-in definition (the AL/JS contract) 8

3.1 Properties . . . 8

3.2 Events: browser -> AL (22) . . . 8

3.3 Procedures: AL -> browser (29) . . . 10

4. Lifecycle: from opening the page to the first picture 11

4.1 Step by step . . . 11

4.2 LoadAllData() in order . . . 11

5. wrapper.js global variables and flags 12

6. BOOT() in detail 13

6.1 Host and container creation (lines 132-163) . . . 13

6.2 Local installer functions (lines 198-554) . . . 13

6.3 DHTMLX plugins and global configuration (lines 557-610) . . . 13

6.4 Templates and loading hook (lines 611-692) . . . 13

6.5 Columns, editors, collapse-all, tooltip (lines 703-890) . . . 13

6.6 Lightbox (lines 892-917) . . . 14

6.7 Task context menu and click handlers (lines 919-1235) . . . 14

6.8 Link tooltip, drag and update events (lines 1237-1485) . . . 14

6.9 Datastores, CSS injection, layout and init (lines 1524-1659) . . . 14

7. JSON contracts (what AL builds, what JS expects) 15

7.1 Task (codeunit 50613 CreateJobTaskJsonObject -> LoadProjectData) . . . 15

7.2 Dependency link (codeunit 50616 GetLinksAsJson -> LoadLinksData) . . . 15

7.3 Day Planning (codeunit 50613 BuildDayPlanningJsonObject -> LoadDayPlanningsData) . . . 15

7.4 Resource, holiday and background-task result . . . 16

8. How the data is built in AL (codeunit 50613) 17

8.1 GetDateRange - the visible window . . . 17

8.2 GetJobTasksAsJson - the task tree . . . 17

8.3 Resources and Day Plannings . . . 17

9. The task panel (task list + timeline) 18

9.1 Rows and bars . . . 18

9.2 Date handling rules (important) . . . 18

9.3 Header features . . . 18

10. The resource panel 19

10.1 Layout: two stacked rows sharing one time axis . . . 19

10.2 Panel state on the AL side . . . 19

10.3 Two scopes . . . 19

10.4 How the timeline is drawn (renderResourceLine) . . . 19

10.5 Header controls . . . 20

10.6 Hover tooltips . . . 20

11. Background-task pipeline and the poll loop 21

11.1 Why each safeguard exists . . . 21

12. Right-click menus, clicks and double-clicks 22

12.1 Task row / task bar menu . . . 22

12.2 Resource name menu (right-click a name in the resource grid) . . . 22

12.3 Day marker menu (right-click a green/red/grey marker) . . . 22

12.4 Empty resource timeline cell menu . . . 22

12.5 Left click and double-click . . . 23

13. Editing flows: from the browser back into Business Central 24

13.1 Dragging or resizing a task bar . . . 24

Period-sync scenarios (codeunit 50617 CalculateChanges) . . . 24

13.2 Inline cell edit and lightbox . . . 24

13.3 Dependency links . . . 24

13.4 Add Day Planning, Undo, Redo, Add Marker . . . 24

14. Refresh and navigation: Refresh Data, Previous, Today, Next 26

14.1 RefreshGantt - the full reload . . . 26

14.2 Previous / Today / Next . . . 26

14.3 Everything that triggers a full RefreshGantt . . . 26

14.4 After the Day Plannings / Sequences page is closed . . . 26

15. Gantt Settings and what each setting changes 28

15.1 Company-wide visual settings (Daily Optimizer Setup) . . . 28

16. Page 50620 actions: purpose of every menu entry 29

16.1 Other interactions that are not ribbon actions . . . 29

17. Known quirks and observations 30

18. Performance design and measurements **31**

18.1 What was optimised 31

18.2 Measured on the NL_Test sandbox (834 Day Plannings, 118 resources) 31

18.3 Remaining hot spots to watch with large data 31

Appendix A. wrapper.js function index **32**

Appendix B. Page 50620 procedures and triggers **34**

Appendix C. Troubleshooting guide **35**

Appendix D. Glossary **36**

1. Purpose, scope and file inventory

This document explains, in detail, how Business Central page **50620 'Gantt Demo DHX 2'** works together with the control add-in **'DHX Gantt Control 2'**, its JavaScript wrapper (**wrapper.js**) and the **DHTMLX Gantt** component (**dhtmlxgantt.js**). It follows the data from the moment the page opens, through the control add-in, into the JavaScript, and into the DHTMLX widgets, and then follows user interaction back from the browser into AL triggers.

It is written for developers who need to maintain or extend the Gantt, and for consultants who need to understand what each menu, button and setting does and what is happening behind it.

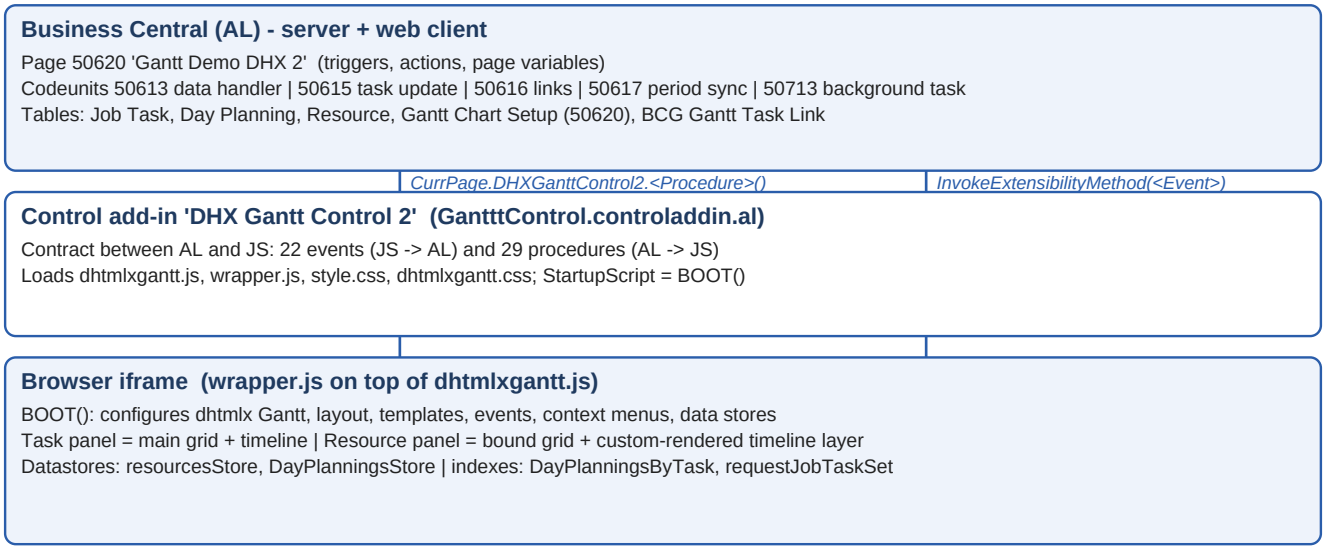
1.1 What the page does

- Shows **Job Tasks** (Job No. + Job Task No.) as a hierarchical task list with a timeline of task bars (the **task panel**).
- Shows **resources** and their **Day Plannings** per day in a second panel under the Gantt (the **resource panel**): hours per resource per day, requested (grey) versus assigned (green/red) load.
- Lets the user drag/resize task bars (which updates the Job Task planned dates and, after a preview, the dependent Day Plannings), draw dependency links, and open the related BC pages (Job Task card, Day Plannings, Sequences, Resource card, Resource calendar, Task Scheduler, Summary).
- Supports a configurable visible period (per-user 'Gantt Settings') and Previous / Today / Next navigation.

1.2 Source files

File	Lines	Role
`src/dhx/ganttdemo2/page_50620_GanttDemo.al`	~2000	The BC page. Hosts the usercontrol, implements every trigger raised by the add-in, owns the actions/ribbon, the page state (AnchorDate, filters, panel flags) and the background-task plumbing.
`src/dhx/ganttdemo2/GantttControl.controladdin.al`	95	Control add-in definition: scripts, styles, startup script, the 22 events and 29 procedures that form the AL/JS contract.
`src/dhx/ganttdemo2/wrapper.js`	~3650	All client-side logic. BOOT() configures DHTMLX; the rest are AL-callable functions, renderers, event handlers and helpers.
`src/dhx/ganttdemo2/startupScript.js`	1	Single line: BOOT(). Runs when BC has loaded the add-in scripts.
`src/dhx/ganttdemo2/style.css`	322	Gantt look and feel: weekend/holiday shading, bar styles, collapse icon, tooltips, bold rows.
`src/dhx/dhtmlxgantt.js / .css`	vendor	The DHTMLX Gantt library (with auto_scheduling, marker, undo, resource and tooltip plugins used here).
`codeunit_50613_GanttChartDataHandler.al`	~1170	Builds every JSON payload sent to the add-in: tasks, resources, day plannings (paged and incremental), holidays; GetDateRange.
`codeunit_50615_GanttUpdateData.al`	263	Applies a dragged/edited task from the browser back to the Job Task (UpdateJobTaskFromJson, ApplyPlannedDatesOnly).
`codeunit_50616_GanttChartLinkHandler.al`	409	CRUD of dependency links (table 'BCG Gantt Task Link') and constraint helpers.
`codeunit_50617_DayPlanningPeriodSyncMgt.al`	600	When a task period changes: computes which Day Plannings move, shows the preview page, applies the move.
`codeunit_50713_GanttBGResourcePanelData.al`	147	Page Background Task that builds resource panel JSON off the interactive session.
`codeunit_50614_GanttBCPageHandler.al`	36	Opens the correct Job Task card by 'Job View Type'.
`page_50650_GanttChartSetup.al + table 50620`	123	'Gantt Settings' card and its per-user setup table.
`page_50662 / table_50661`	-	DayPlanning Period Sync Preview page and its temporary buffer table.

2. Architecture overview

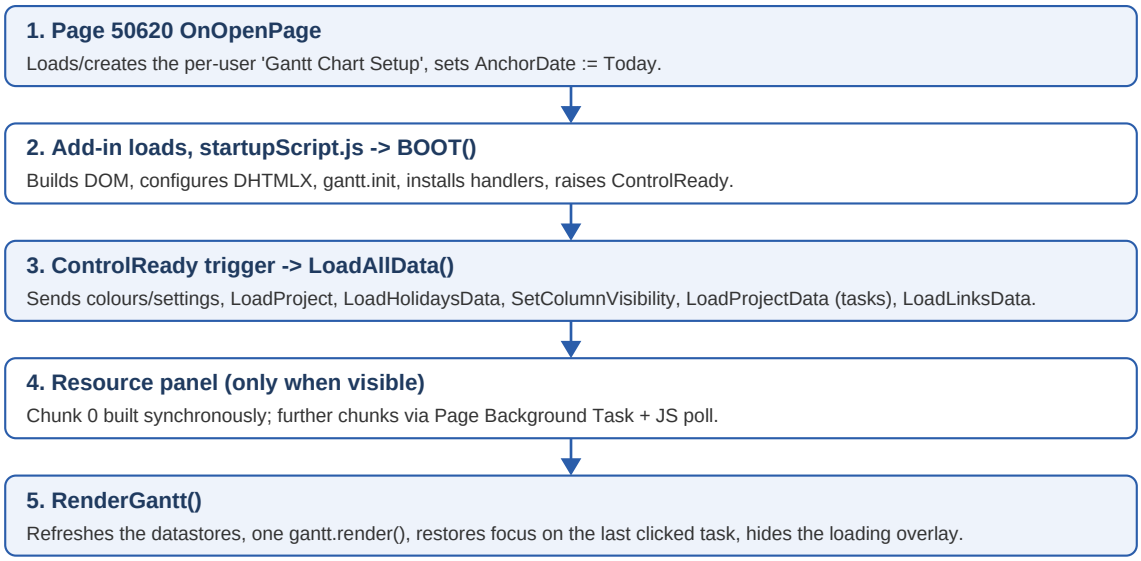


The control add-in is an iframe inside the BC web client. BC executes the scripts listed in the add-in definition inside that iframe. Because **wrapper.js is a classic script (not a module)**, every top-level `function` it declares becomes a global and is therefore callable from AL as `CurrPage.DHXGanttControl2.<Name>(...)` as long as the same name is declared as a `procedure` in the control add-in file. In the other direction, JavaScript raises AL triggers with `Microsoft.Dynamics.NAV.InvokeExtensibilityMethod('<EventName>', [args])`; the page implements each one as `trigger <EventName>(...)` under `usercontrol(DHXGanttControl2; ...)`.

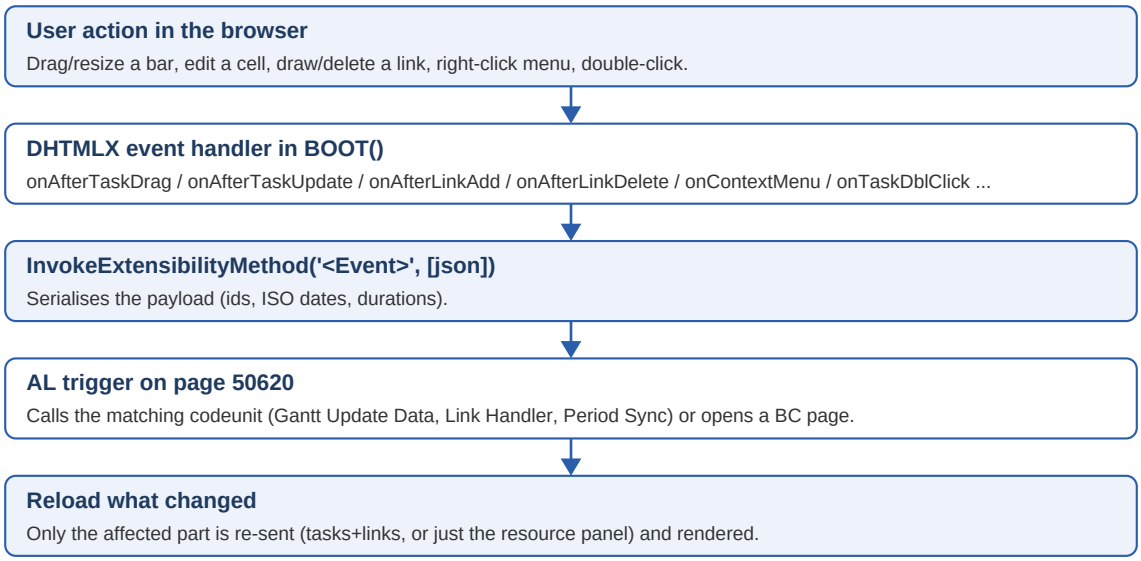
2.1 Three rules of this integration

1. **All data is JSON text.** AL builds Text payloads (JsonArray/JsonObject.WriteTo) and passes them to JS; JS parses with `_tryParseJson` and feeds DHTMLX. Inbound edits travel the other way as JSON strings in a single Text argument.
2. **Ids are composite strings.** A Job Task is `JobNo|JobTaskNo` (pipe), a resource is `RES-<No>`, a Day Planning id is the record SystemId. Because the DHTMLX task id equals the BC key, a task event can be mapped back to a Job Task with `Split('|')` and no lookup table.
3. **Add-in calls are not allowed from background-task completion triggers.** BC rejects any `CurrPage.<control>.X()` issued from OnPageBackgroundTaskCompleted/Error. The solution is a JS-driven poll (section 11) that pulls pending results through a normal synchronous trigger.

2.2 End-to-end data flow (read path)



2.3 End-to-end data flow (write path)



3. The control add-in definition (the AL/JS contract)

File: `GantttControl.controladdin.al` (control add-in **'DHX Gantt Control 2'**). It has no logic; it is the interface that both sides code against.

3.1 Properties

Property	Value	Meaning
RequestedHeight / MinimumHeight	`1500 / 600`	The add-in asks for a tall area because the resource panel is stacked under the task panel; VerticalStretch/Shrink let BC fit it to the window.
RequestedWidth / MinimumWidth	`1800 / 700`	Wide timeline; HorizontalStretch/Shrink adapt to the window.
Scripts	`dhtmlxgantt.js, wrapper.js`	Order matters: the library first, then the wrapper that uses the global `gantt` object.
StyleSheets	`dhtmlxgantt.css, style.css`	Library skin first, then BC-specific overrides.
StartupScript	`startupScript.js`	Contains only `BOOT();`. BC runs it when the add-in frame is ready.

3.2 Events: browser -> AL (22)

Each event is raised from wrapper.js and handled by a trigger of the same name on page 50620.

Event	Arguments	Raised when / what the AL trigger does
`ControlReady`	-	End of BOOT(). Page sets ResourcePanelFlag := false, calls LoadAllData() then SetResourcePanelVisibility.
`OnAfterInit`	-	Raised by BC after the add-in is created. Ensures the per-user setup record exists (setup.EnsureUserRecord / Get).
`onTaskDblClick`	eventId, eventData	Double-click on a task row/bar, or menu 'Show Task'. Splits the id and opens the Job Task card via codeunit 50614 (card depends on 'Job View Type').
`onOpenDayPlanning`	taskId, eventData	Menu 'Show Day Planning'. Opens page 'Day Plannings' filtered to the Job/Task and the bar's visual period.
`onOpenDayPlanningVisual`	taskId, eventData	Menu 'Show Task Scheduler'. Opens page 'DHX Scheduler (Project)' for the Job/Task.
`OnJobTaskUpdated`	eventData (JSON)	A task bar was dragged/resized or edited. Calls Gantt Update Data.UpdateJobTaskFromJson, shows the Day Planning period-sync preview, then reloads tasks/links/panel (section 13).
`OpenResourceLoadDay`	resourceId, taskDate, planStatus, idList	Double-click or menu 'Open Day Plannings' on a resource-panel day marker. Opens Day Plannings for that resource/day, then refreshes the panel only if data changed.
`OpenSequences`	idList	Menu 'Open Sequences' on a marker. Opens page 'Day Planning Sequence' with the related sequence rows highlighted (modal), then refreshes the panel if data changed.
`OnLinkCreated / OnLinkDeleted`	linkData (JSON)	User drew or removed a dependency arrow. Stored/removed by Gantt Chart Link Handler (UpsertLinkFromJson / DeleteLinkFromJson).
`OnShowResourcesForTask`	taskId, childrenJson, periodFrom, periodTo	Task-bar left click or menu 'Show Job Resources'. Builds the task-scope (task + children), shows the resource panel and loads it.
`OnShowSummaryForTask`	taskId, childrenJson, periodFrom, periodTo	Menu 'Summary'. Opens page 'Summary View' for the Job/Task.
`OnResourceDblClick`	resourceId	Double-click or menu 'Show Resource Card' on a resource name. Opens the Resource Card.
`OnResourceHover`	resourceId	Mouse over a resource name. Computes per-resource statistics for the visible period and returns them via SetResourceStats.
`onOpenResourceScheduler`	resourceId	Menu 'Resource Calendar'. Lets the user pick a week (StrMenu) then opens 'Resource Scheduler - Calendar' for that resource/week.
`OnResetResourceFilter`	-	Red (x) next to the resource-panel header. Clears the task scope and reloads the unfiltered panel.

Event	Arguments	Raised when / what the AL trigger does
`onAddDayPlanning`	resourceId, taskDate	Menu 'Add Day Planning' on an empty resource-timeline cell. Pre-fills a new Day Planning (resource, date, work-hour template) and opens the card.
`OnResourceFilterRetrieved`	filterJson	Answer to GetResourceFilter: stores the JS-side filter JSON in CurrentResourcePanelFilterJsonString.
`OnGanttFilterIconClick`	-	Funnel icon in the 'Task name' header. Opens the 'Task Scheduler Filter' dialog; applies Job/Job Task filter and refreshes.
`OnGanttClearTaskFilter`	-	Red (x) next to the funnel. Clears JobFilter/JobTaskFilter and refreshes.
`OnGanttContextAddFilter`	jobNo, jobTaskNo	Menu 'Add Filter' on a task row. Applies that row's Job/Task (range for parents) as the filter, no dialog.
`OnPollResourcePanelResult`	-	Poll tick (section 11). Pushes any pending background-task result into the add-in and chains the next chunk.

3.3 Procedures: AL -> browser (29)

Procedure	Arguments	JS function and purpose
`LoadProject`	start, end (Date)	LoadProject(): sets gantt.config.project_start/end and start_date/end_date = visible window; configures auto_scheduling (schedule_on_parse:false). Shows the loading overlay.
`LoadProjectData`	jsonText	LoadProjectData(): clears tasks/links, gantt.parse({data,links}) inside gantt.silent(); sets the _reloadInProgress guard.
`LoadLinksData`	linksJsonTxt	Adds every dependency link (silently), then renders.
`LoadHolidaysData`	holidaysJsonTxt	Fills _ganttHolidays {date: description}; re-renders so holiday shading appears.
`LoadResourcesData`	resourcesJsonTxt	resourcesStore.clearAll()+parse(); renders unless a full refresh is in progress.
`LoadDayPlanningsData`	json, IsFirstChunk, HasMore	Builds DayPlanningsByTask/requestJobTaskSet indexes and parses the chunk into DayPlanningsStore; renders on the last chunk.
`ApplyDayPlanningChanges`	changedJson, resourcesJson	Incremental patch of only the changed Day Plannings after a sub page closes; renders once.
`ClearData`	-	Starts a full reload: sets _isRefreshing, clears indexes, stores, tasks, links.
`RenderGantt`	skipTrigger	Final step of every reload: refresh stores, gantt.render(), restore focus, reset _isRefreshing, hide overlay.
`SetColumnVisibility`	ShowStartDate, ShowDuration, ShowTaskType	Shows/hides grid columns (see the argument-mapping caveat in section 17).
`SetResourcePanelVisibility`	toggle	RecreateGanttLayout(toggle): swaps the layout with/without the resource panel.
`SetResourcePanelFilterInfo / ClearResourceFilter / GetResourceFilter`	job, task, from, to	Maintain _resourceFilterInfo (shown in the header tooltip) and report it back via OnResourceFilterRetrieved.
`SetGanttTaskFilterInfo`	job, task, from, to	Maintains _ganttTaskFilterInfo and updates the funnel/(x) buttons in the task-name header.
`SetBarFontColor / SetDayOffColors / SetGanttTaskBarDefaults / SetTooltipColors`	colours / sizes	Apply per-company visual settings as CSS custom properties and gantt.config.task_height.
`SetResourceStats`	resourceId, statsJson	Caches statistics for the resource-name hover tooltip.
`UpsertLink / DeleteLink`	json / id	Add-in helpers to patch a single link.
`Undo / Redo / AddMarker`	- / date,text	gantt.undo(), gantt.redo(), and add a vertical date marker.
`GetGanttData`	-	JS alert showing project_start/project_end (diagnostic).
`NotifyResourcePanelTaskPending / StopResourcePanelPolling`	-	Start/stop the bounded poll loop for background-task results.
`RefreshEventData`	eventData	Declared in AL but has no JavaScript implementation; only referenced from dead code (section 17).

4. Lifecycle: from opening the page to the first picture

4.1 Step by step

1. **Page opens** (`OnOpenPage`). `OptiSetup.Get()` reads Daily Optimizer Setup. The per-user `Gantt Chart Setup` row is read or created (default: Calculated range, From formula '-CW', To formula 'CW+4W', all three Load flags true). `AnchorDate := Today()`, `ShowPreviousNext` is true unless the range type is a fixed 'Date Range'.
2. **BC creates the add-in iframe** and loads dhtmlxgantt.js, wrapper.js (top-level code only declares variables/functions), the CSS files, then runs startupScript.js.
3. `BOOT()` (wrapper.js:132) runs once (`_booted` guard): creates the host and `#gantt_here` containers, shows the loading overlay, checks the library, installs plugins and configuration, defines templates/columns/lightbox/menus/events, creates the two datastores, builds the layout with `RecreateGanttLayout(true)`, calls `gantt.init('gantt_here')`, installs the document-level tooltip/menu watchers, and finally raises **ControlReady**.
4. `ControlReady` **trigger** sets `ResourcePanelFlag := false` (panel starts hidden so nothing is fetched for it), runs `LoadAllData()` and then hides the resource panel layout.
5. `LoadAllData()` (section 4.2) sends visual settings, the period, holidays, column settings, tasks, links and calls `RenderGantt(false)`.
6. `RenderGantt` calls `gantt.render()` once, hides the overlay. The user sees the task panel.

4.2 LoadAllData() in order

`LoadAllData` is the single procedure behind first open, Refresh Data, Previous/Today/Next, Gantt Settings and filter changes. The order is deliberate:

#	AL call	JS function	Why here
1	`SetBarFontColor(GetBarFontColor)`	`SetBarFontColor`	Text colour of bar labels (CSS variable --bar-font-color).
2	`SetDayOffColors(weekend, holiday)`	`SetDayOffColors`	Colours of .weekend/.holiday cells (--gantt-weekend-color/--gantt-holiday-color).
3	`SetGanttTaskBarDefaults(border, progress, font, size, height)`	`SetGanttTaskBarDefaults`	Global bar look: border colour, progress colour override, font, gantt.config.task_height.
4	`SetTooltipColors(bg, font)`	`SetTooltipColors`	Hover popups (task tooltip, marker tooltip, filter popups).
5	`GetDateRange(Setup, AnchorDate, Start, End)`	-	Resolves the visible window from the setup record (section 8.1).
6	`SetGanttTaskFilterInfo(JobFilter, JobTaskFilter, start, end)`	`SetGanttTaskFilterInfo`	Funnel/(x) state in the task-name header.
7	`LoadProject(Start, End)`	`LoadProject`	Sets project_start/end and the displayed time scale window; shows the overlay.
8	`LoadHolidaysData(Start, End)`	`LoadHolidaysData`	Base Calendar non-working days for shading.
9	`SetColumnVisibility(...)`	`SetColumnVisibility`	Applies the setup's column switches.
10	`LoadTaskData() when 'Load Job Tasks'`	`LoadProjectData`	The task tree (codeunit 50613.GetJobTasksAsJson).
11	`LoadLinkData()`	`LoadLinksData`	Dependency arrows.
12	`GetResourceFilter()`	`GetResourceFilter`	Keeps the JS-side filter copy in sync (not used for branching).
13	`Resource panel branch`	`see section 10`	If a task scope is stored: SetResourcePanelFilterInfo + filtered reload; else if the panel is visible: default reload.
14	`SetResourcePanelVisibility(true) when visible`	`RecreateGanttLayout`	Layout with the resource panel.
15	`RenderGantt(false)`	`RenderGantt`	The one real paint; ends the refresh cycle.

Note: A native `Dialog` ('Loading Gantt data... Job Tasks... Links...') is shown during LoadAllData. Control add-in calls only execute in the browser after the whole AL trigger returns, so a JS overlay cannot paint while the server is still building JSON; the Dialog is the only live progress indicator in that window.

5. wrapper.js global variables and flags

These are the module-level variables at the top of wrapper.js (and a few that are created later). Understanding them explains most of the control flow.

Variable	Type / initial	Purpose and where it is used
`gantt_here`	DOM element / null	The #gantt_here container DHTMLX renders into. Used as the 'initialised' test by almost every AL-callable function (`!gantt_here || !gantt.$root` -> ignore the call).
`_booted`	bool / false	Makes BOOT() idempotent.
`_queuedColumnArgs`	array / null	If AL calls SetColumnVisibility before init finished, the args wait here and are applied right after gantt.init.
`resourcesStore`	DHTMLX datastore	Rows of the resource panel (`{key,label}` -> id=key). Bound to the resource grid and timeline ('bind: resources').
`DayPlanningsStore`	DHTMLX datastore	All loaded Day Plannings (one item per Day Planning). Read by the resource renderer, 'Total Hours' column and tooltips.
`window.DayPlanningsByTask`	map taskId -> [DayPlanning]	Index used to draw the black request bars on task bars and to find a task's Day Plannings. Rebuilt on the first chunk, appended on later chunks, patched by ApplyDayPlanningChanges.
`window.requestJobTaskSet`	map taskId -> true	Tasks that have at least one Request Day Planning; makes the task bar bold (class bc-task-bold).
`_isRefreshing`	bool / false	True between ClearData() and RenderGantt(). While true, the Load* functions skip their own gantt.render() so the full cycle paints once.
`_resourceFilterInfo`	{job,task,periodFrom,periodTo} / null	Task-scope of the resource panel; drives the blue (i)/red (x) buttons and tooltip in the panel header.
`_resourcePanelInvokeInFlight`	bool	Serialises the three round trips into the panel (task-bar click, reset button, poll). A second call while one is outstanding would wedge BC's call queue ('Working on it...').
`_ganttTaskFilterInfo`	object / null	Job/Job Task filter of the task list; drives the funnel and (x) buttons in the 'Task name' header.
`_ganttHolidays`	map date -> text	Non-working Mon-Fri days from the Base Calendar; used by scale_cell_class/timeline_cell_class.
`_gtbProgressOverride`	string	Setup colour for the progress fill. Empty = derive a darker shade of the bar colour per task (`_darkenHex(color, 0.60)`).
`skipTrigger_OnJobTaskUpdated`	bool / false	RenderGantt(true) sets it so the next programmatic task update does not raise OnJobTaskUpdated again (prevents a save loop).
`_allTasksCollapsed`	bool / false	State of the collapse/expand-all icon in the 'Task name' header.
`_lastClickedTaskId`	id / null	Last task bar clicked. After a reload, `_restoreTaskFocusAfterReload()` calls showTask/selectTask so the view returns to it.
`_loadingOverlay / _loadingSafetyTimer`	DOM / timer	The LOADING overlay and its 180 s safety timeout (never stay stuck if RenderGantt is not reached).
`_dragInProgress`	bool (BOOT scope)	True 300 ms after a drag; blocks the onAfterTaskUpdate events produced by the drag itself or by auto-scheduling cascades.
`_reloadInProgress`	bool (BOOT scope)	True while LoadProjectData runs (and 500 ms after); blocks phantom onAfterTaskUpdate events from a data load.
`_resourcePanelPollTimer / Attempts / Active`	timer / int / bool	State of the bounded poll loop that fetches background-task results.
`RESOURCE_PANEL_POLL_INTERVAL_MS / _FIRST_MS / _MAX_ATTEMPTS`	1500 / 400 / 20	Poll cadence: first tick after 400 ms, then every 1.5 s, at most 20 ticks (about 30 s) per burst.
`_filterToolbarWatcherBusy`	bool	Re-entrancy guard of the MutationObserver that re-injects filter buttons.
`resourceLayers`	[renderResourceLine, 'taskBg']	Layers of the resource timeline: the custom renderer plus the standard background grid.

6. BOOT() in detail

`window.BOOT` (wrapper.js lines 132-1659) is one long function that runs once. It is easiest to read as the following blocks, in source order.

6.1 Host and container creation (lines 132-163)

Finds BC's host element (`#controlAddIn`, or `document.body`), sizes it 100%x100%, creates `#gantt_here` inside it, builds the loading overlay (`_createLoadingOverlay`) and shows it immediately - this is the earliest moment any JS runs, long before the first AL call. If the DHTMLX library is missing it logs an error and stops.

6.2 Local installer functions (lines 198-554)

These nested functions are defined first and executed at the end of BOOT (each guarded so it installs once per document):

Function	Purpose
`InstallResourceMarkerCustomTooltipsForDayPlannings`	Document-level mousemove/mouseout capture on `.gantt_resource_marker`. On hover it looks up every Day Planning in `DayPlanningsByTask` with the same resource_id and work_date and shows a tooltip table (Project, Task, Status, Skill, Seq, Requested Start/End/Idle/Hours, Assigned Start/End/Idle/Hours; first 8 rows). Resource label comes from resourcesStore.
`InstallResourceGridDblClick`	Double-click on a `.res-name-cell` raises OnResourceDblClick (opens the Resource Card).
`InstallResourceNameHoverTooltip`	On hover over a resource name it asks AL (OnResourceHover) for statistics once per 30 s per resource; AL answers with SetResourceStats, rendered as a Statistic/Value table (skills, total Day Plannings, requested/assigned/capacity hours).
`InstallResourceGridContextMenu`	Builds three body-level menus and the capture-phase contextmenu handler: (a) resource name menu, (b) marker menu, (c) empty-cell menu (section 12). Also closes menus on mouse leave, click elsewhere and Escape.

6.3 DHTMLX plugins and global configuration (lines 557-610)

- `gantt.plugins({auto_scheduling, marker, undo, resource, tooltip})`: auto-scheduling (dependency/constraint recalculation while dragging), vertical date markers, Undo/Redo, resource views, hover tooltips.
- `row_height` default 30, `task_height` default 13 (overridden later by SetGanttTaskBarDefaults).
- `date_format = xml_date = '%Y-%m-%d'`: ISO dates everywhere between AL and JS.
- `scales`: two header rows - **week** ('wk 45 (Mon, 02 Nov 2026 - Sun, 08 Nov 2026)', alternating week-even/week-odd CSS) and **day** ('Mon 02').
- `resource_property = 'resource_id'`: the Day Planning field that links a Day Planning to a resource row.
- `work_time = false`, `min_column_width = 55`: durations are calendar days (matches BC's Integer Duration); with work_time=true dragging across a weekend would stretch bars.

6.4 Templates and loading hook (lines 611-692)

Hook	What it does
`templates.scale_cell_class / timeline_cell_class`	Adds `weekend` (Sat/Sun) or `holiday` (in _ganttHolidays) to header and body cells; colours come from CSS variables.
`templates.task_class`	Adds the scheduling type class, `bc-task-bold` when the task has Request Day Plannings (requestJobTaskSet), `bc-baseline-bar` for non-Posting tasks (task.bold) and `bc-posting-thin-bar` for Posting tasks.
`templates.progress_text`	Shows the progress percentage (as a number) on the bar when above 0.
`onTaskLoading`	Runs for every task parsed: normalises progress 0-100 -> 0-1; sets task.color (default #3b8ef0) and task.progressColor; makes non-Posting tasks `readonly`; calls `_normalizeTaskBarDates`. Colour is deliberately not stored elsewhere because task.color is DHTMLX's native hook.

6.5 Columns, editors, collapse-all, tooltip (lines 703-890)

- **Editors**: text (description), date (start), number (duration, 0+), number (progress 0-100), select (constraint type: asap, alap, snet, snlt, fnet, fnlt, mso, mfo), date (constraint date).

- **Columns** (`gantt.config.columns`): `text` (tree, 250 px, with the collapse/expand-all SVG icon in its header), `progress` (%), `start_date`, `duration`, `constraint_type`, `constraint_date`, `schedulingType` ('Task Type'), `add` (+ button). Visibility is controlled by SetColumnVisibility.
- **onGridHeaderClick** toggles ToggleCollapseExpandAll() when the icon is clicked (sets `$open` on every task and re-renders).
- **grid_row_class** makes non-Posting rows bold (`bc-row-bold`).
- **tooltip_text** builds the task hover tooltip: Job and Task table (job description = the top-level ancestor row), Period (start and inclusive end = DHTMLX exclusive end minus one day), and optional constraint rows (Constraint, Constraint Date, Max Duration) from the BC business-constraint fields (`bcConstraintType`, `bcConstraintDate`, `bcMaxDuration`).

6.6 Lightbox (lines 892-917)

The DHTMLX edit dialog sections: description (textarea), constraint, scheduling type (Fixed Duration / Fixed Units / Fixed Work), time (duration). The page also blocks the default lightbox on double-click (`onTaskDblClick` returns false) because BC's own Job Task card is opened instead.

6.7 Task context menu and click handlers (lines 919-1235)

`_showContextMenu(x, y, taskId)` builds the right-click menu on a task row/bar. `onContextMenu` shows it, `onTaskClick` turns a left click on a bar into 'Show Job Resources'. The menu items and the AL behind each are described in section 12.

6.8 Link tooltip, drag and update events (lines 1237-1485)

Event	Behaviour
`onGanttReady`	Installs mouseover/mousemove/mouseleave listeners on `gantt.$root` to show a Dependency tooltip (From, To, Type FS/SS/FF/SF, Lag).
`onTaskDblClick`	Raises onTaskDblClick(id, JSON) to open the Job Task card; returns false to block the lightbox.
`onTaskDrag`	While dragging, redraws the black placeholder-date overlay bars (`_renderRequestBars`).
`onAfterTaskDrag`	Sets `_dragInProgress` for 300 ms, re-normalises start_date, `gantt.refreshTask`, redraws overlays, then raises **OnJobTaskUpdated** with {id, bcJobNo, bcJobTaskNo, text, start_date, end_date, duration, schedulingType, constraint_*} unless skipTrigger_OnJobTaskUpdated is set.
`onAfterTaskUpdate`	Same payload for non-drag edits (lightbox, inline cell editor). Ignored while `_dragInProgress` or `_reloadInProgress`.
`onAfterAutoSchedule`	After the engine recalculated dates for dependent tasks, re-pins start_date to midnight for the touched tasks.
`onAfterLinkAdd / onAfterLinkDelete`	Raise OnLinkCreated / OnLinkDeleted with {id, source, target, type, lag}.

6.9 Datastores, CSS injection, layout and init (lines 1524-1659)

1. `gantt.createDatastore({name:'resources'})` and `({name:'DayPlannings'})` with `initItem` that assigns ids (resource id = `item.key`; Day Planning id = `item.id`).
2. Style injection: resource marker colours (`gantt_resource_marker_ok` green, `_overtime` red, `_request` grey), header colours matching BC list headers, filter-icon and tooltip popup styles, and the bar fix that hides DHTMLX's thin planned bracket and forces the actual bar solid.
3. `RecreateGanttLayout(true)` then `gantt.init('gantt_here')` - the only gantt.init call in the program.
4. Apply queued column settings; run the four Install* functions and `_installGanttFilterToolbarWatcher()`.
5. `onGanttRender`: after every render re-injects the resource-header buttons, the task-header filter buttons and the request overlay bars.
6. `InvokeExtensibilityMethod('ControlReady', [])` - tells AL that calls are now safe.

7. JSON contracts (what AL builds, what JS expects)

7.1 Task (codeunit 50613 CreateJobTaskJsonObject -> LoadProjectData)

`LoadProjectData` accepts an array of tasks (or `{tasks, links}`). One object per Job Task:

Field	Source in BC	Used by JS for
`id`	'JobNo|JobTaskNo'	DHTMLX task id; mapped back to the Job Task by Split('|') in every trigger.
`text`	Posting: 'JobTaskNo - Description', otherwise Description	Row label and bar text. The prefix is stripped again before saving (see 13.1).
`start_date`	Effective start (VisualBarStartDate, else Constraint Calc. - Start Date)	Bar start, ISO yyyy-mm-dd.
`duration`	End - Start + 1 recomputed (NOT Job Task.Duration)	Inclusive day count; end_date = start + duration (exclusive).
`bcJobNo / bcJobTaskNo`	Job No. / Job Task No.	Echoed back in events; used by menus and filters.
`schedulingType`	'fixed_duration' / 'fixed_work' / ''	CSS class on the bar, Task Type column, lightbox select.
`bcConstraintType / bcConstraintDate / bcMaxDuration`	Job Task constraint fields (empty unless Constraint Date set)	Task tooltip rows only. Separate from DHTMLX's own constraint_* keys.
`progress`	Progress / 100	Progress fill (JS also accepts 0-100 and converts).
`color`	GetGanttTaskBarColor (Posting) or ...NonPosting	task.color; progressColor derived in JS.
`indentation / bold / open / parent`	Indentation; Job Task Type <> Posting; true for non-Posting; parent task id	Tree: parent is looked up from an array keyed by indentation (ParentJobTaskId); non-Posting rows are expanded, bold and read-only.

7.2 Dependency link (codeunit 50616 GetLinksAsJson -> LoadLinksData)

Field	Meaning
`id`	Composite id built from source/target/type (unique, reconstructable).
`source / target`	Task ids ('JobNo|TaskNo'); both must belong to the same job.
`type`	'0' Finish-to-Start, '1' Start-to-Start, '2' Finish-to-Finish, '3' Start-to-Finish.
`lag`	Lag (Days).

7.3 Day Planning (codeunit 50613 BuildDayPlanningJsonObject -> LoadDayPlanningsData)

One object per Day Planning record. About 610 bytes each. The 'Request' variant (`CreateDayPlanningJsonObjectRequest`) is used for undated requests and sets `work_date = placeholder_date = the Job Task's Placeholder Date` and `plan_status = 'Request'`.

Field	Meaning
`id`	Day Planning SystemId (unique store key).
`task / jobNo / jobTaskNo`	'JobNo-JobTaskNo' (dash, informational), and the two keys; the task index uses jobNo + '|' + jobTaskNo to match the DHTMLX task id.
`dayNo / dayLineNo`	Plan Date and Day Line No. (primary key part).
`work_date / placeholder_date`	ISO date shown in the panel / placeholder date for undated requests.
`start_time / end_time / hours`	Assigned times; hours = Assigned Hours if not zero else Requested Hours (this is the number drawn in the marker).
`requested_* / assigned_*`	start_time, end_time, idle_minutes, hours for both sides (tooltip table).
`resource_id`	'RES-' + Assigned Resource No., else 'RES-' + Requested Resource No., else 'RES-' (unassigned row '- NONE -').
`type / vendorNo`	'Resource'; vendor number or 'null'.
`plan_status`	Request (In Request), Assigned (In Progress), Rejected, Accepted.

Field	Meaning
`work_order_no / skill / sequence_no`	Order Intake No., Skill, Sequence No.

7.4 Resource, holiday and background-task result

Payload	Shape
`Resource`	[{ key: 'RES-<No>', label: 'Name (No)' }] plus a leading '- NONE -' entry (key 'RES-') for unassigned work. Stored with id = key.
`Holiday`	[{ date: 'yyyy-mm-dd', description, type:'holiday' }] - Mon-Fri non-working days only (weekends are shaded by JS).
`Background task result`	Dictionary: resourcesJson, dayPlanningsJson, dayPlanningsHasMore ('true'/'false'), dayPlanningsNextCursor (keyset cursor 'pass|job|task|line').
`Resource filter`	{ job, task, periodFrom, periodTo } - JS state mirrored into AL by GetResourceFilter / OnResourceFilterRetrieved.
`Task update (browser -> AL)`	{ id, bcJobNo, bcJobTaskNo, text, start_date, end_date (exclusive), duration, schedulingType, constraint_type, constraint_date }.

8. How the data is built in AL (codeunit 50613)

8.1 GetDateRange - the visible window

Every load and every report starts from `GetDateRange(GanttSetup, AnchorDate, StartDate, EndDate)`:

Date Range Type	Start	End
Date Range	Setup 'From Date'	Setup 'To Date'
Weekly	Monday of AnchorDate's week	Sunday of that week + 4 weeks (5 weeks in total)
Calculated	CalcDate(From Data Formula, AnchorDate)	CalcDate(To Data Formula, AnchorDate). Both formulas are mandatory (TestField).

If End is blank it becomes 31-12-9999; if Start > End then End := Start. `AnchorDate` is a page variable that Previous/Today/Next change.

8.2 GetJobTasksAsJson - the task tree

1. Reads the per-user setup; Job No. filter = page JobFilter if set, else the setup's 'Job No. Filter'; plus the optional Job Task filter.
2. Opens **two** server-side filtered cursors on Job Task: (1) tasks whose Planned/Visual-bar dates overlap the window, (2) tasks whose persisted 'Constraint Calc. - Start/End Date' overlap the window.
3. **Merge-joins** the two cursors in primary-key order (CompareJobTaskKey); a row in both is processed once. This avoids scanning the whole Job Task table.
4. Skips 'Total' and 'End-Total' rows; `GetEffectivePeriod` picks Planned dates if both set, else the constraint-derived dates; rows with no valid period or out of window are skipped.
5. `CreateJobTaskJsonObject` writes the object (7.1). The hierarchy is built with an array `ParentJobTaskId[Indentation+1]` that is reset on every new Job No.

8.3 Resources and Day Plannings

- **GetResourcesAsJson(Anchor, Job, Task)** collects the distinct resources used by the Day Plannings in the window (Query 'Day Planning Used Resources' plus the undated-request pass), then emits only those resources (plus '- NONE -').
- **GetDayPlanningsAsJson(..., Cursor, PageSize, HasMore, NextCursor)** is the paged variant: keyset paging on the primary key (Job No., Job Task No., Day Line No.) in pass 1 (dated rows in the window) and pass 2 (In Request rows with blank Plan Date whose Job Task Placeholder Date is in the window). Each call resumes with `Find('>')` after the cursor key - no rescanning.
- **GetDayPlanningsByJobTaskAsJson / GetResourcesByJobTaskAsJson** (task-scoped panel) use a Query joined on the list of 'JobNo|JobTaskNo' keys of the task and its children.
- **GetChangedDayPlanningsAsJson** returns only rows with SystemModifiedAt > a timestamp plus their resources (incremental refresh, section 14.4). It returns false when the counts show a delete or a move in/out of scope.
- **GetHolidaysAsJson** uses the Base Calendar from Daily Optimizer Setup and Calendar Management (IsNonworkingDay), skipping Sat/Sun.

9. The task panel (task list + timeline)

The task panel is the upper layout row: a **grid** (left) and a **timeline** (right) that share one vertical scrollbar (`scrollVer`), with a horizontal scrollbar (`scrollHor`) shared with the resource timeline so both panels scroll together in time.

9.1 Rows and bars

- Rows are the Job Tasks of the window, indented by Indentation. Non-Posting rows (headings) are bold, expanded by default, **read-only** and drawn as thin baseline-style bars. Posting rows are the editable, slimmer rectangular bars.
- Bar label = `text`; progress fill shows the percentage; bar colour = setup colour (Posting vs non-Posting), progress colour = darker shade or the setup override.
- Tasks that have at least one **Request** Day Planning get `bc-task-bold`; Request Day Plannings with a placeholder date draw a **black overlay bar** (`.bc-req-bar`) at that date on the task row (`_renderRequestBars`, re-run after every render and during drag).
- Weekends and holidays are shaded in the header and body; the weekly header alternates two tints.
- Dependency links are arrows; hover shows From/To/Type/Lag; drawing a link raises OnLinkCreated, deleting raises OnLinkDeleted.

9.2 Date handling rules (important)

- DHTMLX `end_date` is **exclusive** (start + duration days). BC dates are **inclusive**. AL subtracts one day on the way in (`CalcDate('<-1D>')` in UpdateJobTaskFromJson); JS subtracts one day for tooltips and for the Show Resources/Summary period.
- `_normalizeTaskBarDates` forces `start_date` to 00:00:00 after load, drag, edit and auto-scheduling so bars start exactly on a day boundary. It deliberately leaves end_date alone: an earlier version stamped 23:59:59 on end_date and made bars overshoot by one day.
- `LoadProject` sets `auto_scheduling.schedule_on_parse:false`. The visible window changes on every Previous/Next, so an on-load auto-schedule would pull ASAP tasks to the window start. Auto-scheduling still works for interactive drag/link changes.
- `gantt.silent()` wraps data loads so that parse/render does not fire per-task update events at BC.

9.3 Header features

- **Collapse/Expand all** icon (two SVGs) in the 'Task name' header toggles `$open` for every task.
- **Funnel filter button** (injected after the icon by `_updateGanttFilterToolbar`): grey when no filter, blue when filtered; hover popup shows 'Job = ..., Task = ..., period'; click raises OnGanttFilterIconClick. A red (x) appears when filtered and raises OnGanttClearTaskFilter.
- A **MutationObserver** (`_installGanttFilterToolbarWatcher`) re-injects the buttons if DHTMLX re-renders the header (for example when the panel resizer is dragged) and wipes them.

10. The resource panel

10.1 Layout: two stacked rows sharing one time axis

`RecreateGanttLayout(show)` writes `gantt.config.layout`. With the panel it has two rows: the task row (gravity 2) and the resource row (gravity 1) separated by a resizer; the resource row contains a **grid** (`resourceGrid`, columns 'Name' and 'Total Hours') and a **timeline** (`resourceTimeline`) both **bound to the datastore** `resources`, plus their own vertical scrollbar `resourceVScroll`. Both timelines use the single `scrollHor`, so scrolling in time moves both panels. Without the panel the layout contains only the task row. `gantt.resetLayout()` applies a changed layout and then `_restoreTaskFocusAfterReload()` re-anchors the last clicked task.

10.2 Panel state on the AL side

Page variable	Meaning
`ResourcePanelFlag`	True while the panel is visible. False after ControlReady (panel starts hidden).
`ResourcePanelJobNo / JobTaskNo / ChildTaskIds / From / To`	Task scope when the panel was opened from a task ('Show Job Resources'). Empty = unfiltered 'Default' mode.
`CurrentResourcePanelFilterJsonString`	JSON copy of the JS-side filter; used by Add Day Planning and the Day Plannings action. Can lag (async), so reload logic uses the native variables instead.
`ResourcePanelTaskId / ResourcePanelTaskRunning`	Id of the newest background task and a busy flag (new requests are dropped while a chunk chain is running).
`PendingResourcesJson / PendingDayPlanningsJson / PendingResultAvailable / PendingDayPlanningsHasMore / NextCursor / IsFirstChunk`	A delivered-but-not-yet-pushed background result.
`ResourcePanelChunkJobFilter / JobTaskFilter / AnchorDate`	Scope snapshot so a multi-chunk load keeps one consistent scope even if the user changes filters meanwhile.

10.3 Two scopes

Mode	Opened by	Content	How it is loaded
Filtered (task scope)	Left click on a task bar, menu 'Show Job Resources', reload after a task drag	Resources and Day Plannings of the clicked task and its children for the view window	Built **synchronously** in the page (EnqueueFilteredResourcePanelReload) - the scope is small. LoadResourcesData + LoadDayPlanningsData(json, true, false).
Default (all)	Action 'Show Resource Panel', red (x) reset, any refresh while the panel is visible	All resources/Day Plannings in the window and Job/Job Task filter	**Chunk 0 synchronously** (resources + first 5000 Day Plannings). If HasMore, the next chunks run as Page Background Tasks (section 11).

10.4 How the timeline is drawn (renderResourceLine)

The resource timeline does not use DHTMLX's built-in resource histogram. It registers a **custom layer** `renderResourceLine(resource, timeline)` that returns one absolutely positioned `<div>` row per resource:

1. `calculateResourceLoad(resource, scale)` sums `hours` per calendar day for that resource, ignoring 'Request' rows and non-working days (`gantt.isWorkTime`).
2. For every day with load a cell is created at `timeline.getItemPosition(resource, dayStart, dayEnd)`. **Green** (`gantt_resource_marker_ok`) when the day total is <= 8 hours, **red** (`_overtime`) when above 8.
3. The hours are centred in the cell. A **count badge** (top-left) appears when more than one Day Planning falls on that resource/day.
4. If a Request exists on the same day as a planned cell, a **black 6 px bar** is added at the bottom of that cell.
5. Days that have only Request Day Plannings get a **grey** box (`gantt_resource_marker_request`) with the requested hours and a badge if several.
6. Each cell carries `data-resource-id`, `data-work-date`, `data-plan-status` and `data-id-list` (pipe-separated Day Planning ids). These data attributes drive tooltips, double-click and the context menu.
7. Double-click on a cell raises **OpenResourceLoadDay** (resource, date, status, id list).

Note: The left grid 'Total Hours' column sums `hours` of all loaded Day Plannings of that resource (Request rows included, because `hours` falls back to Requested Hours). The per-cell value excludes Request rows. The two numbers are therefore not expected to match when Requests exist.

10.5 Header controls

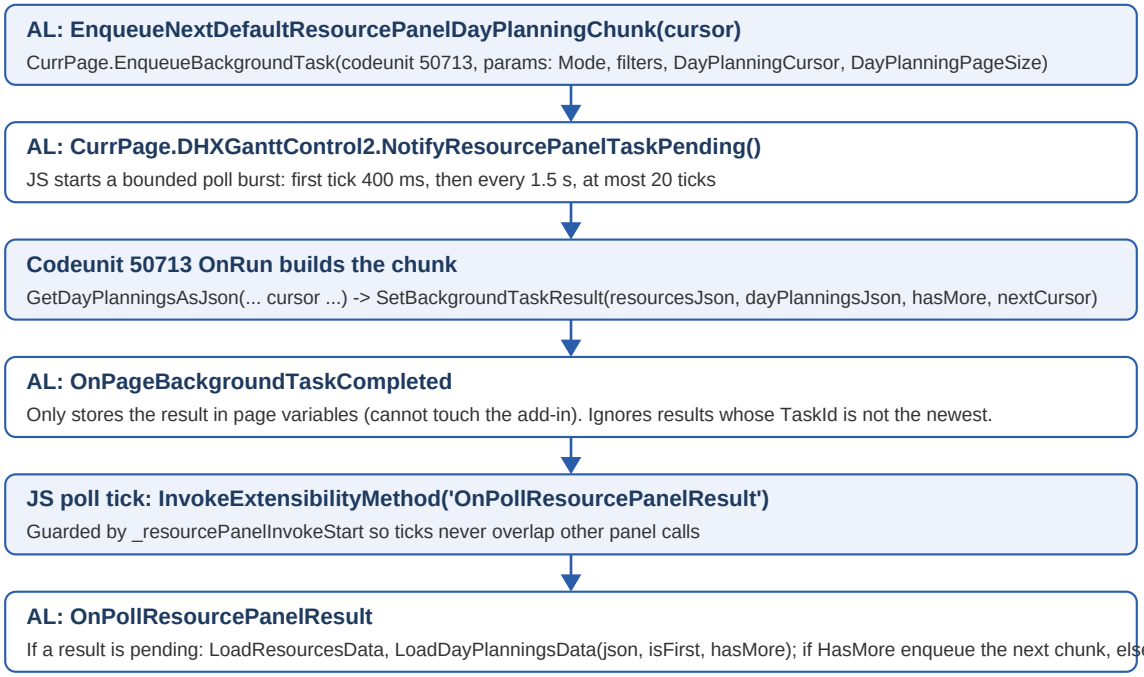
- When a task scope is active, `_updateResourceHeaderTooltip` adds a blue **(i)** button (hover popup: 'Filter applied: Job = ..., Task = ..., period') and a red **(x)** button to the 'Total Hours' header.
- Clicking (x) raises **OnResetResourceFilter**: AL clears the scope and reloads the unfiltered panel. The call is serialised with `_resourcePanelInvokeStart/End`.
- `SetResourcePanelFilterInfo`, `ClearResourceFilter` and `GetResourceFilter` maintain and report `_resourceFilterInfo`.

10.6 Hover tooltips

- **Day marker** hover: table of all Day Plannings of that resource/day with Requested and Assigned Start/End/Idle/Hours (client-side, from DayPlanningsByTask).
- **Resource name** hover: Statistic/Value table (skills, Total Day Plannings, Requested Hours, Assigned Hours, Capacity Hours) computed in AL (`ResScheduler_BuildSectionStatsJson`) for the visible period, cached 30 s per resource.

11. Background-task pipeline and the poll loop

Large Day Planning tables made a single synchronous load block the page, so chunks beyond the first are built by **Page Background Tasks** (codeunit 50713 'Gantt BG Resource Panel Data'), which run in a separate read-only session. BC forbids calling the control add-in from `OnPageBackgroundTaskCompleted`, so a small poll loop in JS pulls the result.



11.1 Why each safeguard exists

Safeguard	Reason
_resourcePanelInvokeInFlight (JS)	BC's call bridge does not block JS while a call is outstanding. A second InvokeExtensibilityMethod before the first returns wedges the queue and every later call hangs ('Working on it...').
ResourcePanelTaskRunning (AL)	EnqueueBackgroundTask has no cancel. Rapid clicking piled up concurrent full-table reads and slowed the whole environment, so new requests are dropped while one runs.
TaskId comparison	A superseded task still completes server-side; its result is discarded if its TaskId is not the newest.
Chunk scope snapshot	Keeps all chunks of one load on the same filters/AnchorDate.
Bounded polling (20 ticks)	Never poll forever on an idle page; each new chunk restarts a fresh burst.
Synchronous chunk 0	A background task costs about 0.8 s (session start + polling) while the query itself is about 0.1 s for ~800 records; the first chunk and the whole task-scoped load therefore run in the page.

12. Right-click menus, clicks and double-clicks

There are **four** context menus plus click/double-click behaviours. All menus close when the mouse leaves them, on click elsewhere, on Escape, and hide hover tooltips while open.

12.1 Task row / task bar menu

Opened by DHTMLX `onContextMenu` on a task. Built by `_showContextMenu`.

Menu item	JS function -> event	What happens in BC
Summary	`_ctxShowSummary -> OnShowSummaryForTask`	Opens page 'Summary View' loaded for the Job/Job Task (LoadDataSet) without the job/task columns.
Show Job Resources	`_ctxShowResources -> OnShowResourcesForTask`	Shows the resource panel scoped to the task and its descendants for the view window; stores the scope in ResourcePanelJobNo/TaskNo/ChildTaskIds; loads synchronously; header (i)/(x) buttons appear.
Show Task	`_ctxOpenTask -> onTaskDblClick`	Opens the Job Task card (Opti Job Task Card for Project view, Job Task Card - Resource for Resource view).
Show Day Planning	`_ctxOpenDayPlanning -> onOpenDayPlanning`	Opens 'Day Plannings' filtered by Job No., Job Task No. and the task's visual bar period.
Show Task Scheduler	`_ctxOpenDayPlanningVisual -> onOpenDayPlanningVisual`	Opens the visual scheduler page 'DHX Scheduler (Project)' for the Job/Task.
Add Filter	`_ctxAddFilter -> OnGanttContextAddFilter`	Sets JobFilter/JobTaskFilter from the row (for a parent row, a 'min..max' Job Task No. range covering its subtree) and runs RefreshGantt; no dialog.
Cancel	-	Closes the menu.

12.2 Resource name menu (right-click a name in the resource grid)

Item	Event	Purpose
Resource Calendar	`onOpenResourceScheduler`	Asks which week of the current Gantt period (StrMenu 'W45: 02-11-2026 - 08-11-2026, ...') and opens 'Resource Scheduler - Calendar' for that resource and week.
Show Resource Card	`OnResourceDblClick`	Opens the Resource Card (only for RES- ids).

12.3 Day marker menu (right-click a green/red/grey marker)

Item	Event	Purpose
Open Day Plannings	`OpenResourceLoadDay`	Opens 'Day Plannings' for that resource and day. Filters: Job/Task of the Day Plannings behind the marker (from the id list, if any), Plan Date = the day, and an **OR group** (FilterGroup(-1)) on Assigned Resource No. = X **or** Requested Resource No. = X (pool resources for VEN- ids). On close, the panel is refreshed only if the data changed (14.4).
Open Sequences	`OpenSequences`	Opens 'Day Planning Sequence' for the Job Task of the first Day Planning, for the Gantt period, highlighting the sequence rows of every Day Planning behind the marker and the sibling Request/Assigned lines of the same resource and day. Modal; same conditional refresh on close.

Note: Why FilterGroup(-1)? In AL, filters on different fields inside group -1 are combined with OR, while all other groups are combined with AND. The marker may show a resource either as the **assigned** or as the **requested** one, so the Day Plannings behind it match when *either* field equals the resource. A plain SetRange on one field would hide the other half. FilterGroup(0) afterwards restores the normal group; the group -1 filters remain active (and are not visible in the page's filter pane).

12.4 Empty resource timeline cell menu

Item	Event	Purpose
Add Day Planning	`onAddDayPlanning`	Computes the clicked day (gantt.dateFromPos) and resource row from the mouse position. AL creates a new Day Planning in status 'In Request' for that resource/date (RES- validates the requested resource; VEN- the vendor), applies the Work-Hour Template (default times, non-working minutes, working hours), takes Job/Job Task from the panel filter, opens 'Day Planning Card - New Record' and on OK validates and inserts it, then reloads (RefreshGantt).

12.5 Left click and double-click

Gesture	Behaviour
Left click on a task bar	Records `_lastClickedTaskId` and runs the same logic as 'Show Job Resources' (opens/refreshes the task-scoped panel). Ignored right after a drag and when the click is on the progress handle or a grid row.
Double-click on a task row/bar	onTaskDblClick -> opens the Job Task card; DHTMLX lightbox is suppressed.
Double-click on a day marker	OpenResourceLoadDay (same as 'Open Day Plannings').
Double-click on a resource name	OnResourceDblClick -> Resource Card.
Click on the collapse/expand icon	Collapses or expands every task.
Drag a bar / its handles	Move or resize the task, drag the progress handle, or drag from a bar end to another bar to create a link (section 13).

13. Editing flows: from the browser back into Business Central

13.1 Dragging or resizing a task bar

Dragging moves the whole task (Scenario: shift), dragging the right edge changes the end, the left edge changes the start. Non-Posting (heading) rows are read-only.

1. JS `onAfterTaskDrag`: blocks follow-up update events for 300 ms, normalises start_date, repaints, and raises **OnJobTaskUpdated** with the JSON payload (end_date is DHTMLX's exclusive end).
2. AL trigger `OnJobTaskUpdated` calls `Gantt Update Data.UpdateJobTaskFromJson`. It reads the Job Task, captures the old Visual Bar Start/End, then applies each field: text (the 'TaskNo - ' prefix is stripped), start_date, end_date (**minus one day**), duration, schedulingType.
3. If the end date changed it is **snapped forward to the next working day** (`SnapForwardToWorkDay`: Work-Hour Template + Base Calendar) so a bar can never end on a weekend/holiday.
4. If the period changed, `HandleJobTaskPeriodChange` -> `CalculateChanges` works out which Day Plannings must move, and opens the **'DayPlanning Period Sync Preview'** page. Nothing is written until the user clicks **Apply Changes**.
5. On Apply, `ApplyChanges` extends the Job Task end if a cascade overshoots, `JobTask.Modify`, moves or inserts Day Plannings, and `Commit`s. The page then reloads tasks + links + the resource panel and calls `RenderGantt(true)`.
6. If the user closes the preview without applying, the page offers a Confirm: update **only** the Job Task planned dates (`TryApplyPlannedDatesOnly`, no Day Planning is touched) or revert the bar. Either way the page reloads so the bar returns to the true server position.

Period-sync scenarios (codeunit 50617 CalculateChanges)

Scenario	Condition	Effect on Day Plannings
1.1 Shift left/right	OldStart <> NewStart and OldEnd <> NewEnd	Every Day Planning moves by the same offset; active-day rules and collisions are resolved forward-only.
1.2 Right bar change	Start same, End changed	Enlarged: new entries for new active days. Reduced: overflowing rows are moved to the nearest free active day, backward from the new end.
1.3 Left bar change	Start changed, End same	Mirror of 1.2 (forward from the new start).

Active days come from the Job Task's Work-Hour Template (else the setup's template) and the Base Calendar (mandatory). Resources with a matching 'Day Planning Pattern' are redistributed onto the pattern's active days; others use the naive offset + forward cascade. Rows are reused (Plan Date only), never deleted/recreated.

13.2 Inline cell edit and lightbox

Editing the text, start date, duration, progress or constraint cells (or the lightbox) fires `onAfterTaskUpdate`, which raises the same **OnJobTaskUpdated** event and follows the same path. Events caused by data loads (`_reloadInProgress`) or by drag cascades (`_dragInProgress`) are ignored so BC is not called for changes that came from BC itself.

13.3 Dependency links

- Drawing a link between two bars raises OnLinkCreated; AL `UpsertLinkFromJson` stores it in table 'BCG Gantt Task Link' (key Job No., Source, Target, Link Type) with Lag (Days). Cross-job links and self-links are rejected (the page shows 'Failed to save link').
- Deleting an arrow raises OnLinkDeleted; AL `DeleteLinkFromJson` deletes the record.
- Links are loaded for the Job filter (or all jobs when none) on every refresh and always sent, even as '[]', so removed arrows disappear.

13.4 Add Day Planning, Undo, Redo, Add Marker

- **Add Day Planning** (empty resource cell menu): see 12.4. On success it ends with a full `RefreshGantt()`.
- **Undo / Redo** (ribbon): call `gantt.undo()` / `gantt.redo()` on the DHTMLX undo stack (the undo plugin is enabled). Behaviour is DHTMLX's; it was not separately verified against BC persistence.

- **Add Marker**: adds a vertical marker via `AddMarker(date, text)`. The ribbon action currently passes a fixed demo value ('2024-06-15', 'New Marker').

14. Refresh and navigation: Refresh Data, Previous, Today, Next

14.1 RefreshGantt - the full reload

```
procedure RefreshGantt()
begin
    CurrPage.DHXGanttControl2.ClearData();   // JS: _isRefreshing := true; clear indexes, stores, tasks, links
    Setup.Get(UserId);                       // pick up Gantt Settings changes
    LoadAllData();                           // section 4.2 ... ends with RenderGantt(false)
end;
```

In JS, `ClearData` refuses to run twice in parallel (`_isRefreshing`). While the flag is set, `LoadResourcesData`/`LoadDayPlanningsData` only fill the stores and skip their own render. `RenderGantt` is the single paint: it calls `store.refresh()` (forces bound views to re-read), `gantt.render()`, `_restoreTaskFocusAfterReload()` (showTask/selectTask on the last clicked task), then clears `_isRefreshing` and hides the overlay.

14.2 Previous / Today / Next

These three actions are visible only when `ShowPreviousNext` is true, i.e. when the Date Range Type is **not** the fixed 'Date Range'. They only change **AnchorDate** and then call `RefreshGantt()`:

Action	New AnchorDate	Notes
Today	Today()	Re-centres the window on the current date.
Previous	Weekly: AnchorDate - 5 weeks. Calculated: AnchorDate - (length of the window in days).	Step equals the window width, so windows tile with no gap or overlap.
Next	Weekly: + 5 weeks. Calculated: + window length.	Window length = (To formula date - From formula date + 1), from GetPeriodLength.

What shifts with the window: the time scale (`LoadProject` resets project_start/end), the task query (only tasks overlapping the new window are loaded), holidays, links, and the resource panel (the panel period follows the displayed period; a task-scoped panel is rebuilt for the new period, and a default panel reloads for the new window). Because the window changes on every click, `schedule_on_parse` is disabled so loading never auto-schedules tasks against a moving boundary. Right and left movement in the *timeline* itself (horizontal scrollbar) does not call AL: the whole window is already loaded.

14.3 Everything that triggers a full RefreshGantt

Trigger	Why a full reload
Refresh Data, Today, Previous, Next	New data window or user request.
Gantt Settings closed (RunModal)	Reload with the saved settings (period, loads, columns, filter).
Funnel filter applied/cleared, 'Add Filter'	Task query changes.
Add Day Planning saved	New row added (still a full reload).
Task drag (OnJobTaskUpdated)	Not RefreshGantt: reloads tasks + links + resource panel and RenderGantt(true) (13.1).
Day Plannings / Sequences closed	Not RefreshGantt any more (14.4).

14.4 After the Day Plannings / Sequences page is closed

Previously both flows ended with a full `RefreshGantt()` (about 3.2 s on the test data) even when nothing changed. Now:

1. Before opening the sub page, AL takes a **fingerprint**: the number of Day Plannings in the visible window (plus undated requests) and their latest `SystemModifiedAt` (`GetDayPlanningFingerprint`).
2. After the page closes, `RefreshResourcePanelIfChanged` takes it again. **Unchanged: nothing is sent and nothing is rendered.**
3. Changed, Default-mode panel, no chunk load running: `GetChangedDayPlanningsAsJson` returns only the records modified after the old timestamp (and their resources) provided that `BeforeCount + created = AfterCount`; the JS `ApplyDayPlanningChanges` patches the store and the two indexes and renders **once**.

4. Otherwise (a delete, a record moved in/out of the period, a task-scoped panel, a chunk chain still running): the resource panel alone is reloaded (stored-filter reload or default reload). Tasks, links and holidays are never reloaded.

Measured (834 Day Plannings, NL_Test)	Before	After
Close with no change	about 3.2 s (full reload)	no work
Close after editing one Day Planning	about 3.2 s	about 0.35 - 0.46 s
Result check	-	store, task index, request set and resource count identical to a full Refresh Data (0 differences)

15. Gantt Settings and what each setting changes

Action **Gantt Settings** opens page 50650 (card on table 50620 'Gantt Chart Setup', one row per user, created on first use) modally. When it closes, the page runs `RefreshGantt()`, so every change is visible immediately.

Setting	Effect on the Gantt
Job No. Filter	Default Job filter for tasks, links and Day Plannings (overridden by the page-level filter from the funnel / Add Filter). Blank = all jobs.
Date Range Type = Date Range	Fixed window From Date..To Date. Previous/Today/Next are hidden.
Date Range Type = Calculated	Window = CalcDate(From Date Formula, Anchor) .. CalcDate(To Date Formula, Anchor). Default '-CW' .. 'CW+4W'. Navigation steps by the window length.
Date Range Type = Weekly	Anchor's week plus 4 more weeks (5 weeks). Navigation steps 5 weeks.
Load Job Tasks	When off, LoadTaskData is skipped: no task bars (links and panel still follow their own flags).
Load Resources	Passed as pLoadResources to the default panel reload (resource rows).
Load Day Plannings	Passed as pLoadDayPlannings to the default panel reload (markers and request overlays).
Show Start Date / Show Duration / Show Task Type	Grid column switches sent via SetColumnVisibility. See the argument-mapping caveat in section 17.
Download Data for Inspection	After loading tasks, offers to download the task JSON (GanttProjectTaskData.json) for troubleshooting.

15.1 Company-wide visual settings (Daily Optimizer Setup)

These are not in the Gantt Settings page but are re-applied on every LoadAllData, so a change takes effect on the next refresh:

Setting (via codeunit 50609 getters)	JS call	Effect
Bar Font Color / GTB Font Color	SetBarFontColor, SetGanttTaskBarDefaults	Text colour of on-bar labels (CSS --bar-font-color).
Weekend Color / Holiday Color	SetDayOffColors	Shading of weekend and holiday columns.
GTB Color / GTB Color (non posting)	task JSON 'color'	Fill of Posting vs heading bars.
GTB Border Color / Progress Color / Font Size / Height	SetGanttTaskBarDefaults	Bar border, progress fill (blank = auto darker), label size, bar height (gantt.config.task_height).
Tooltip Background / Font Color	SetTooltipColors	All hover popups.
Base Calendar	LoadHolidaysData	Non-working days shaded as holidays; also used by period sync.
Work hour Template	-	Default times for Add Day Planning and active days for period sync.

16. Page 50620 actions: purpose of every menu entry

The ribbon is built from `area(Processing)`, `area(Navigation)`, `area(Reporting)` and the `area(Promoted)` layout. Direct promoted entries are flat buttons; the others are grouped as **Export**, **Related**, **Check**, **Reports**.

Action (caption)	Group	What it does
Gantt Settings	flat	Opens the Gantt Settings card modally, then RefreshGantt().
Undo / Redo	flat	gantt.undo() / gantt.redo().
Add Marker	flat	Adds a vertical marker (fixed demo date/text).
Clear Gantt Data	flat	Calls ClearData(): empties tasks, links and stores in the browser (no reload; use Refresh Data to bring data back).
Refresh Data	flat	RefreshGantt(): full reload of the current window.
Previous / Today / Next	flat	Move AnchorDate and reload (section 14.2). Hidden for a fixed 'Date Range'.
Get JSON Tasks / Resources / Day Plannings Data	Export	Download the corresponding JSON payload as a file (same builders as the Gantt uses) - useful to inspect what is sent to the add-in.
Summary	Related	Opens 'Summary View' for the visible period.
Day Plannings	Related	Opens 'Day Plannings' for the visible period; when a task-scoped resource panel is active it uses that Job/Task/period, otherwise the Job filter.
Projects	Related	Opens 'Opti Job List' filtered by the Job filter.
Project Tasks	Related	Opens 'Job Task List - Project' for tasks overlapping the window (excluding Total/End-Total), filtered by the Job filter.
Show Resource Panel / Hide Resource Panel	Related	Show: sets ResourcePanelFlag, shows the (empty) panel, clears any task scope and loads the default panel. Hide: hides the panel (no data is discarded).
Check Gantt Data	Check	JS alert with project_start/project_end (diagnostic).
Check Page Period	Check	Message with the computed start/end for the anchor date.
Check Resource Panel Filter	Check	Fetches the JS-side filter and shows its JSON.
Show Resources for Task	hidden	Placeholder; the real work is done by the right-click event OnShowResourcesForTask.
Day Resource Detail	Reports	Report 'Day Resource Details' for the visible period.
Day Overview	Reports	Report 'Day Planning Details' for the visible period.
Week Overview	Reports	Report 'DayPlanning' (week overview) for the visible period.
Daily Capacity Balance Report	Reports	Report 'Daily Capacity Balance Report' for the visible period.

16.1 Other interactions that are not ribbon actions

- **Funnel filter / (x)** in the 'Task name' header: filter dialog ('Task Scheduler Filter' report page, Job No. and Job Task No.) / clear.
- **Collapse/expand all** icon in the 'Task name' header.
- **Red (x) in the resource header**: reset the task scope and reload the unfiltered panel.
- **Hover**: task tooltip, link tooltip, marker tooltip, resource statistics.
- **Resizers** between task and resource panels and between grid and timeline (the filter buttons self-heal after a resize).

17. Known quirks and observations

Found while reading the code; none of them were changed as part of this document. They are listed so maintainers are not surprised.

Observation	Detail / consequence
SetColumnVisibility argument mismatch	The AL procedure declares 3 parameters (ShowStartDate, ShowDuration, ShowTaskType) but the JS function takes 5 (start, duration, constraint type, constraint date, task type). With 3 arguments the third value lands on 'constraint type' and the last two are undefined, which hides the 'Task Type' and 'constraint date' columns. Align the signatures if the 'Show Task Type' switch is expected to control the Task Type column.
RefreshEventData has no JS implementation	Declared in the add-in and called from onTaskDblClick only when `PossibleChanges` is true, but that variable is never set, so the branch is dead code.
OnProjectDataLoaded is not a declared event	LoadProjectData calls InvokeExtensibilityMethod('OnProjectDataLoaded', ...) although the add-in declares no such event; the call has no AL handler.
Unused JS helpers	UpsertTask, DeleteTask, UpsertDayPlanning, DeleteDayPlanning, highlightDayPlanning exist in wrapper.js but are not declared as add-in procedures, so AL cannot call them (UpsertDayPlanning is superseded by ApplyDayPlanningChanges).
Leftover `debugger;` statements	onAfterTaskDrag and onAfterTaskUpdate contain `debugger;` lines, which pause execution if the browser dev tools are open.
Legacy AL procedure OnJobTaskUpdated	A page procedure of the same name as the trigger exists at the end of the page file (older direct JobTask update); the trigger is the live path.
Add Marker uses fixed values	The ribbon action always adds '2024-06-15' / 'New Marker'.
Add Day Planning refreshes everything	onAddDayPlanning still ends with RefreshGantt() instead of an incremental update.
Total Hours vs cell hours	The grid total includes Request hours; the day cells exclude them (10.4).
Unassigned work	Day Plannings with no resource are grouped under the '- NONE -' row (resource id 'RES-').

18. Performance design and measurements

18.1 What was optimised

- **No full reload after Day Planning sub pages**: fingerprint + incremental patch (14.4).
- **First chunk synchronous**: avoids the ~0.8 s background-task start-up and poll latency; only later chunks use the background task.
- **Keyset paging**: each chunk resumes from the last primary key instead of re-scanning all rows (the old Skip method was O(n) per chunk, O(n squared) overall). Chunk size 5000.
- **Parse each chunk once, render once**: the old code re-parsed the whole accumulated array on every chunk and called gantt.render() per chunk.
- **First poll after 400 ms** instead of 1.5 s.
- **Filtered panel built in the page**: scope is small, so no background task.

18.2 Measured on the NL_Test sandbox (834 Day Plannings, 118 resources)

Operation	Before	After
Close Day Plannings, no change	~3.2 s	0 (nothing sent)
Close Day Plannings, one record edited	~3.2 s	~0.35 - 0.46 s
Right-click task > Show Job Resources	~0.94 s	~0.35 s
Reset to the full panel	~1.7 s	~1.0 s
Server time to build 834 Day Plannings	-	~58 ms (22 ms resources)
Client gantt.render() with the panel	-	~26 ms
Fixed cost per background chunk (9 chunks of ~100 rows)	-	~0.6 s each (so use large chunks)

18.3 Remaining hot spots to watch with large data

- **Payload size**: about 610 bytes per Day Planning (fields are partly duplicated, e.g. start_time and assigned_start_time). At tens of thousands of records the transfer dominates; trimming fields would help but wrapper.js reads many of them.
- **renderResourceLine** loops over *all* Day Plannings for every resource cell (several nested scans per cell). Cost grows with resources x days x Day Plannings. An index by resource and date (built once per render) would make it linear.
- **'Total Hours' column** also scans all Day Plannings per resource row.
- **Marker tooltip** scans DayPlanningsByTask on every mouse move over a marker.
- **Full RefreshGantt** remains the cost of Refresh Data, Previous/Next/Today, filter changes and Add Day Planning.

Appendix A. wrapper.js function index

Line numbers are approximate (they shift as the file changes). 'Nested' functions live inside BOOT().

Function	Line	Purpose
`_resourcePanelInvokeStart / End`	36 / 41	Acquire/release the single in-flight slot for panel round trips.
`_createLoadingOverlay / _showGanttLoading / _hideGanttLoading`	69 / 114 / 124	Build, show (with 180 s safety timer) and hide the LOADING overlay.
`BOOT`	132	One-time initialisation (section 6).
`InstallResourceMarkerCustomTooltipsForDayPlannings (nested)`	198	Marker hover tooltip table.
`InstallResourceGridDblClick (nested)`	327	Resource name double-click -> OnResourceDblClick.
`InstallResourceNameHoverTooltip (nested)`	345	Resource statistics tooltip with 30 s cache; defines window.SetResourceStats.
`InstallResourceGridContextMenu (nested)`	396	Resource name / marker / empty-cell context menus.
`_showContextMenu / _hideContextMenu (nested)`	949 / 1005	Task context menu build/close.
`_ctxOpenTask, _ctxShowSummary, _ctxShowResources, _ctxOpenDayPlanning, _ctxOpenDayPlanningVisual, _ctxAddFilter (nested)`	1032-1178	Menu item handlers; each builds its argument JSON and calls InvokeExtensibilityMethod.
`_stripJobTaskNoPrefix / _findRootAncestorTask (nested)`	866 / 876	Tooltip helpers (task description without 'No - ', job description = root ancestor).
`_renderRequestBars`	1664	Draw black overlay bars for Request Day Plannings with a placeholder date.
`RecreateGanttLayout`	1722	Layout with/without the resource panel; resetLayout; restore focus.
`LoadProject`	1827	Set window and auto-scheduling config; show overlay.
`RenderGantt`	1884	Final render of a reload cycle.
`_restoreTaskFocusAfterReload`	1939	showTask/selectTask on the last clicked task.
`SetColumnVisibility / _applyColumnSettings / _setColumnHidden`	1972-1993	Apply column switches (queued until init).
`Undo / Redo / ToggleCollapseExpandAll / AddMarker`	2007-2024	Simple actions.
`_tryParseJson / _normalizeId`	2041 / 2054	Safe JSON parse; id extraction tolerant of key names.
`_normalizeTaskBarDates`	2112	start_date to midnight (end_date untouched).
`_darkenHex`	2121	Darker shade for progress colour (hex, rgb or any CSS colour via canvas).
`_toGanttDate / _applyTaskPatch`	2162 / 2187	Date conversion; partial task update.
`ClearData`	2210	Begin full reload.
`LoadProjectData`	2243	Load tasks (silent).
`UpsertTask / DeleteTask / UpsertLink / DeleteLink`	2343-2449	Single-item patch helpers (links only are in the add-in contract).
`LoadLinksData`	2466	Load all links.
`calculateResourceLoad`	2496	Hours per resource per working day (Requests excluded).
`renderResourceLine`	2530	Custom resource timeline row renderer (10.4).
`LoadResourcesData`	2729	Replace resource rows.
`LoadDayPlanningsData`	2777	Chunk-aware Day Planning load.
`NotifyResourcePanelTaskPending / _scheduleResourcePanelPoll / _runResourcePanelPoll / StopResourcePanelPolling`	2880-2950	Poll loop (section 11).

Function	Line	Purpose
`UpsertDayPlanning / ApplyDayPlanningChanges / DeleteDayPlanning`	2952-3070	Single patch / incremental batch patch / delete.
`highlightDayPlanning`	3121	Legacy selection highlight (unused).
`_ensureCustomTooltip / _showCustomTooltip / _hideCustomTooltip / _hideAllTooltips`	3129-3196	Shared body-level tooltip.
`GetGanttData`	3198	Diagnostic alert.
`SetResourcePanelVisibility`	3202	Layout toggle.
`SetResourcePanelFilterInfo / GetResourceFilter / ClearResourceFilter`	3221-3241	Panel filter state.
`_updateResourceHeaderTooltip / _positionResFilterTooltip`	3243 / 3331	(i) and (x) buttons and popup.
`SetGanttTaskFilterInfo`	3357	Task-list filter state.
`SetBarFontColor / SetDayOffColors / SetGanttTaskBarDefaults / SetTooltipColors`	3377-3438	Visual settings.
`_installGanttFilterToolbarWatcher / _updateGanttFilterToolbar / _positionGanttFilterTooltip / _escHtml`	3457-3606	Funnel/(x) buttons, MutationObserver self-heal, helpers.
`LoadHolidaysData`	3618	Holiday map + re-render.

Appendix B. Page 50620 procedures and triggers

Name	Kind	Purpose
`OnOpenPage`	page trigger	Load setup, AnchorDate := Today, ShowPreviousNext.
`ControlReady / OnAfterInit`	add-in trigger	Start the first load (panel hidden).
`OnPageBackgroundTaskCompleted / OnPageBackgroundTaskError`	page trigger	Stash background results / show a Notification on failure.
`RefreshGantt`	procedure	Full reload (14.1).
`SetJobFilter / SetJobTaskFilter`	procedure	Let other pages open the Gantt pre-filtered.
`LoadAllData`	local	The load sequence (4.2).
`LoadTaskData / LoadLinkData / LoadHolidaysData / LoadDayPlanningData`	local	One-call wrappers around the data handler + add-in call.
`EnqueueFilteredResourcePanelReload`	local	Task-scoped panel, built synchronously.
`EnqueueDefaultResourcePanelReload`	local	Default panel: chunk 0 synchronous, rest queued.
`EnqueueNextDefaultResourcePanelDayPlanningChunk`	local	Queue chunk N via cursor.
`ReloadResourcePanelFromStoredFilter`	local	Re-derive the period and reload the task-scoped panel (uses native vars).
`BuildResourcePanelJobTaskKeysText`	local	';' separated 'JobNo|JobTaskNo' list for the scope.
`GetDayPlanningFingerprint / RefreshResourcePanelIfChanged`	local	Change detection and conditional incremental refresh (14.4).
`ClearResourcePanelFilter`	local	Reset scope variables and the JS filter.
`CalcNewAnchorDate`	local	Previous/Next anchor maths.
`TryValidateRequestedResourceNo`	TryFunction	Friendly error for Add Day Planning.
`ParseDate / OnJobTaskUpdated (procedure)`	local / legacy	ISO date parser; legacy direct update.
`ResourcePanelDayPlanningChunkPageSize / BoolToParamText`	local	Chunk size (5000) / 'true' & 'false' text (Format(Boolean) would give localised Yes/No).

Appendix C. Troubleshooting guide

Symptom	Where to look
Gantt stays on LOADING	A JS error before RenderGantt; the overlay hides itself after 180 s. Open dev tools console and look for 'LoadProjectData failed' or 'BOOT warning'.
'Working on it...' never clears	Two panel round trips overlapped. Check `_resourcePanelInvokeInFlight` and that every InvokeExtensibilityMethod for the panel uses the 5-argument callback form.
Resource panel empty after refresh	Panel scope variables (ResourcePanelJobNo/TaskNo) vs flag; a background task result discarded by TaskId; Setup 'Load Resources/Day Plannings' off.
Bars shifted by one day	Exclusive vs inclusive end date (9.2); check end_date handling and `_normalizeTaskBarDates`.
Bars jump to the window start on load	auto_scheduling.schedule_on_parse must be false (LoadProject).
Task update fires twice / popup loops	`_dragInProgress` / `_reloadInProgress` / `skipTrigger_OnJobTaskUpdated` / `PreviewCancelled` guards.
Filter buttons disappear after resizing	MutationObserver watcher; check it is installed (`_ganttFilterToolbarWatcherInstalled`).
Marker tooltip shows 'No DayPlannings'	DayPlanningsByTask not rebuilt (first chunk not delivered) or the Day Plannings are Request rows with placeholder dates.
Slow after closing Day Plannings	Fingerprint changed and the fallback path ran (delete/move) - check the count rule in GetChangedDayPlanningsAsJson.
Columns do not match Gantt Settings	SetColumnVisibility argument mismatch (section 17).

Appendix D. Glossary

Term	Meaning
Control add-in	A BC extension point that hosts custom HTML/JS in an iframe on a page.
DHTMLX Gantt	Third-party JavaScript Gantt component (dhtmlxgantt.js).
Datastore	DHTMLX data container (resourcesStore, DayPlanningsStore) that views can be bound to.
Layout / view	DHTMLX layout tree of grid, timeline, scrollbar and resizer views.
Task panel / resource panel	Upper task grid+timeline / lower resource grid+timeline.
Day Planning	BC record: a planned (requested or assigned) resource block on a date for a Job Task.
Request (grey/black)	Day Planning in status 'In Request'; shown grey, excluded from the day load, and as a black bar when it shares a day with planned work or has a placeholder date.
AnchorDate	Page variable the visible window is calculated from.
Fingerprint	Count + latest SystemModifiedAt used to detect changes after a sub page closes.
Keyset cursor	'pass|JobNo|JobTaskNo|DayLineNo' - where the next chunk resumes.
Page Background Task	BC mechanism running a codeunit in a separate read-only session and returning a result dictionary.
FilterGroup(-1)	AL filter group whose field filters are combined with OR.